BUSCA COMPETITIVA

(ADVERSARIAL SEARCH)

Prof. Wagner Igarashi, Dr.

Prof. Wagner Igarashi, Dr.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- ***"Em que examinamos os problemas que surgem quando tentamos planejar com antecedência em um mundo no qual outros agentes estão fazendo planos contra nós".***

(RUSSEL e NORVIG, 2013)

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Serão abordados ambientes competitivos, em que
  - os objetivos dos agentes estão em conflito,
  - dando origem a problemas de busca competitiva (frequentemente conhecidos como jogos).

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- A teoria de jogos (matemática),
  - um ramo da economia,
  - visualiza qualquer ambiente multiagente como um jogo,
  - desde que o impacto de cada agente sobre os outros seja “significativo”,
  - não importando se os agentes são cooperativos ou competitivos.
- Em IA,
  - alguns “jogos” são de um tipo especializado
  - que os teóricos denominam jogos determinísticos
  - de revezamento de 2 jogadores de soma zero com informações perfeitas (como o xadrez).

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- São
  - ambientes determinísticos
  - completamente observáveis
  - em que 2 agentes agem alternadamente
  - em que os valores de utilidade no fim do jogo são sempre iguais e opostos (ou simétricos).
- Ex: se um jogador ganha um jogo de xadrez, o outro jogador necessariamente perde.
- É essa oposição entre as funções utilidade dos agentes que gera a situação de competição.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Para pesquisadores de IA,
  - a natureza abstrata dos jogos os torna um assunto atraente para estudo.
- É fácil representar o estado de um jogo e,
  - em geral,
  - os agentes se restringem a um pequeno número de ações
  - cujos resultados são definidos por regras precisas.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Jogos físicos
  - como críquete e hóquei sobre o gelo,
  - têm descrições mais complicadas,
  - uma faixa maior de ações possíveis e
  - regras imprecisas definindo a legalidade das ações.
  - esses jogos físicos não atraíram muito interesse na comunidade de IA.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Exceção: futebol de robôs.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Jogos são interessantes *porque são difíceis de resolver.*
- *Ex.:*
  - *o xadrez tem um fator médio de* ramificação *35*,
  - as partidas com frequência chegam até a 50 movimentos por jogador
  - a árvore de busca tem aproximadamente de 35^{100} a 10^{154} nós
    - embora o grafo de busca tenha “apenas” cerca de 1040 nós distintos.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Os jogos,
  - exigem a tomada de *decisão,*
  - *mesmo quando o cálculo da decisão ótima é inviável,*
  - e penalizam a ineficiência de forma severa.
- Por essa razão, a busca de jogos elaborou várias ideias interessantes sobre
  - como fazer o melhor uso possível do tempo.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Começamos com uma definição do movimento ótimo e um algoritmo para descobri-lo.
- Em seguida, examinaremos técnicas para escolher um bom movimento quando o tempo é limitado.
- A poda nos permite ignorar partes da árvore de busca que não fazem diferença para a escolha final,
- e as funções heurísticas nos dão
  - uma aproximação da utilidade verdadeira de um estado
  - sem realizar uma busca completa

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

Jogos MIN e MAX (ou Minimax)

- Jogos com 2 jogadores
- MAX faz o 1º movimento,
  - e depois eles se revezam até o jogo terminar.
- No fim do jogo,
  - os pontos são dados ao jogador vencedor e
  - são impostas penalidades ao perdedor.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Um jogo pode ser definido formalmente como uma espécie de problema de busca com os seguintes componentes:
  - S_0*: o estado inicial.*
  - JOGADORES(*s*)*: define qual jogador deve se mover em um estado s.*
  - AÇÕES(*s*)*: retorna o conjunto de movimentos válidos em um estado s.*
  - RESULTADO(*s, a*)*: o modelo de transição que define o resultado de um movimento.*

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- TESTE DE TÉRMINO*(s): um teste de término, que é verdadeiro quando o jogo termina e, do* contrário, falso.
  - Os estados em que o jogo é encerrado são chamados estados terminais.
- UTILIDADE*s(s, p): uma função utilidade (ou função objetivo ou função* compensação) define o valor numérico para um jogo que termina no estado terminal *s por um* jogador *p.*

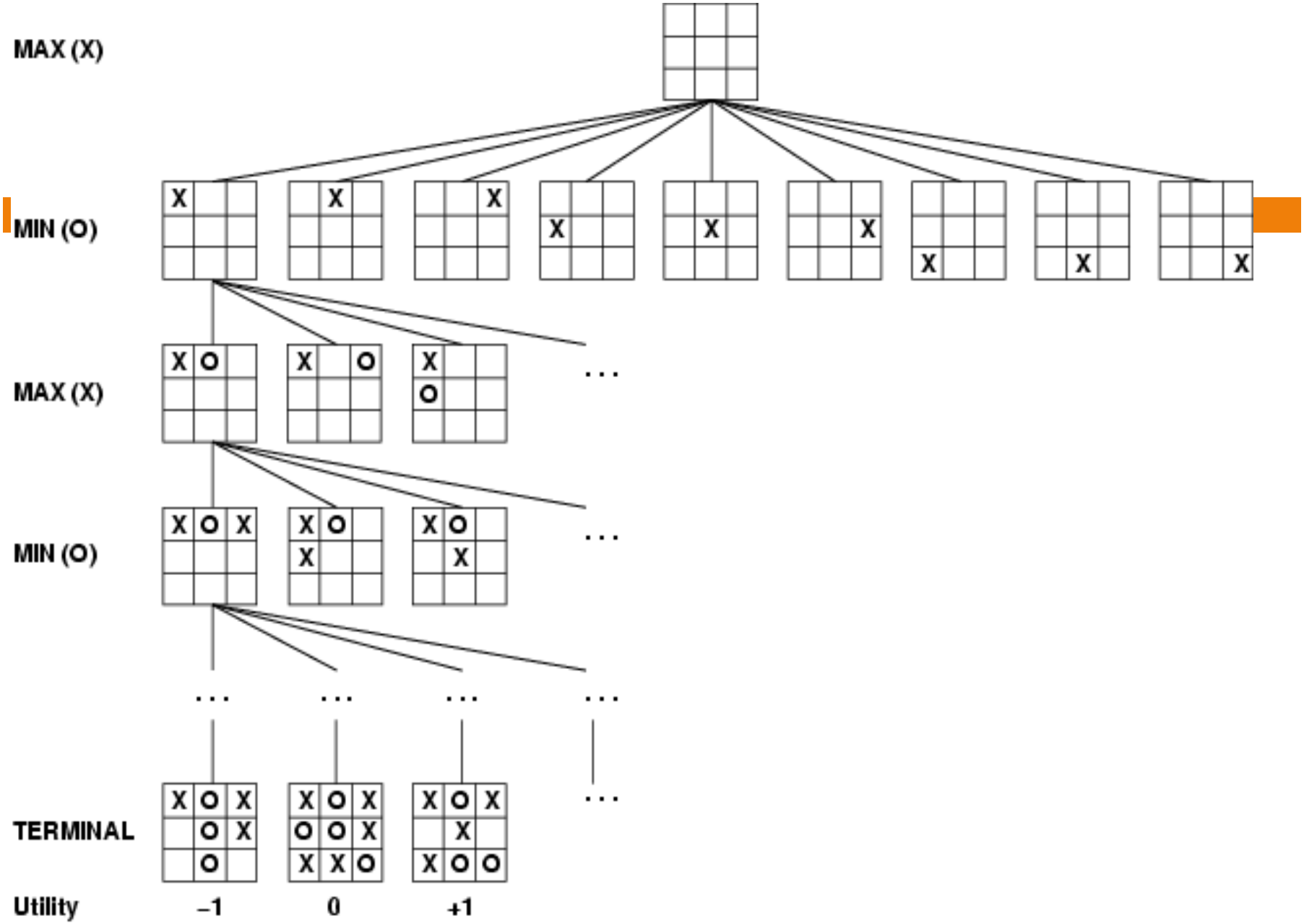
PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- *No xadrez, o resultado é*
  - *uma vitória: +1*
  - *uma derrota: 0*
  - *ou um empate: ½.*
- O xadrez é de soma zero porque cada jogo tem compensação (0 + 1), (1 + 0) ou (½ + ½).
  - "Soma constante" teria sido um termo melhor,
  - mas soma zero é tradicional
  - e faz sentido se você imaginar que de cada jogador é cobrada uma taxa de entrada de ½ ponto.
- Alguns jogos têm uma variedade ampla de resultados possíveis
  - a compensação no gamão varia de 0 até +192.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- No estado inicial as funções AÇÕES e RESULTADO definem a árvore de jogo
  - onde os nós são estados do jogo e
  - as arestas são movimentos.
- A Figura a seguir mostra parte da árvore para o jogo da velha.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com



PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- A partir do estado inicial, MAX tem 9 movimentos possíveis.
- O número em cada nó folha indica o valor de utilidade do estado terminal,
  - do ponto de vista de MAX;
  - valores altos são considerados bons para MAX e ruins para MIN (o que explica os nomes dados aos jogadores).

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Para o jogo da velha, a árvore de jogo é pequena
  - menos de 9! = 362.880 nós terminais.
- Para o xadrez, há mais de 10^{40} nós terminais.
- Independentemente do tamanho da árvore de jogo
  - o trabalho de MAX é a busca de uma boa jogada.
- Usamos o termo árvore de busca
  - para uma árvore que está sobreposta à árvore de jogo completa, examinando os nós o suficiente para permitir que um jogador determine que lance fazer.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

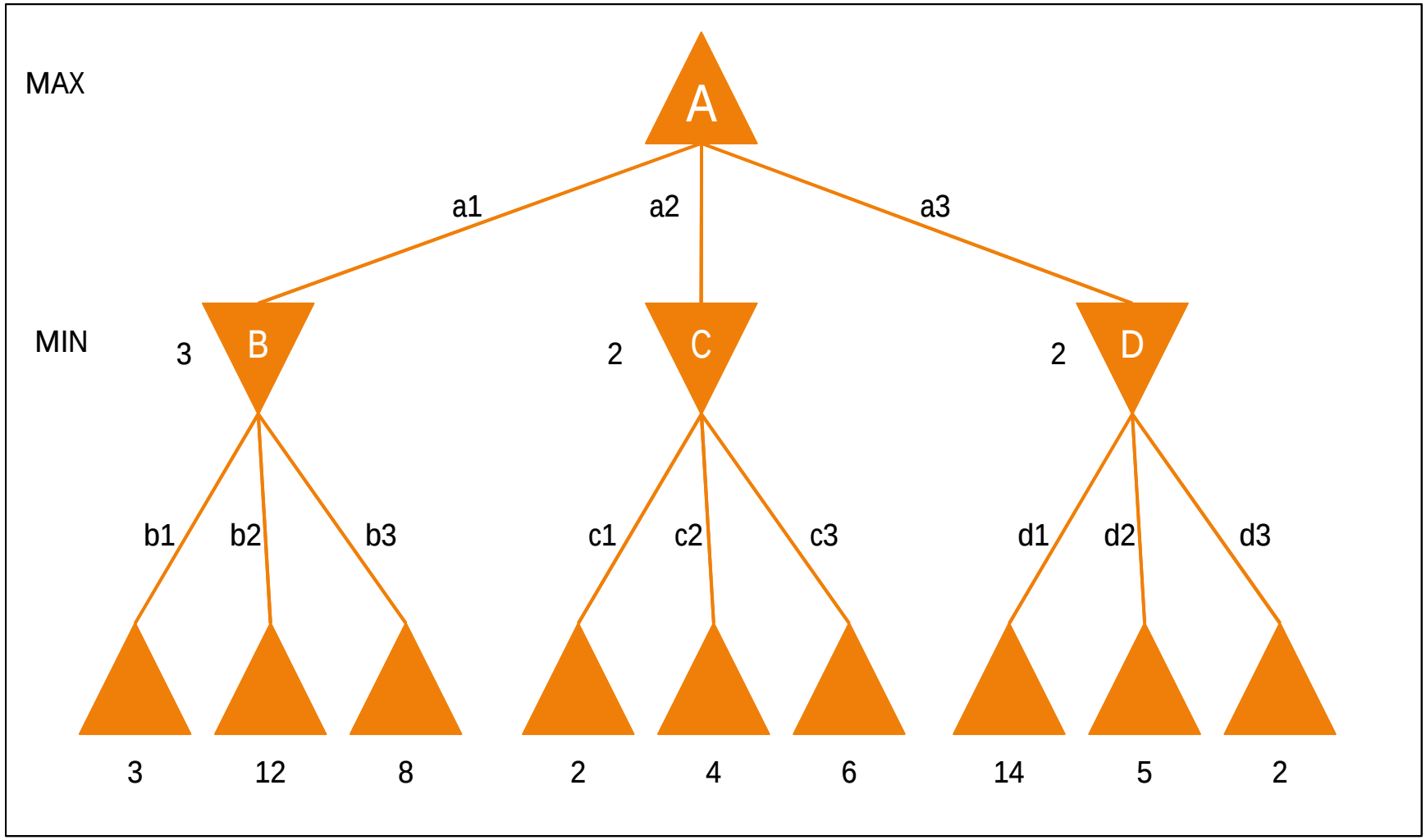
Estratégia ótima

- A solução ótima para MAX depende dos movimentos de MIN, logo:
  - MAX deve encontrar uma *estratégia de contingência* que especifique o movimento de MAX no estado inicial, e depois o movimento de MAX nos estados resultantes de cada movimento de MIN e assim por diante...

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

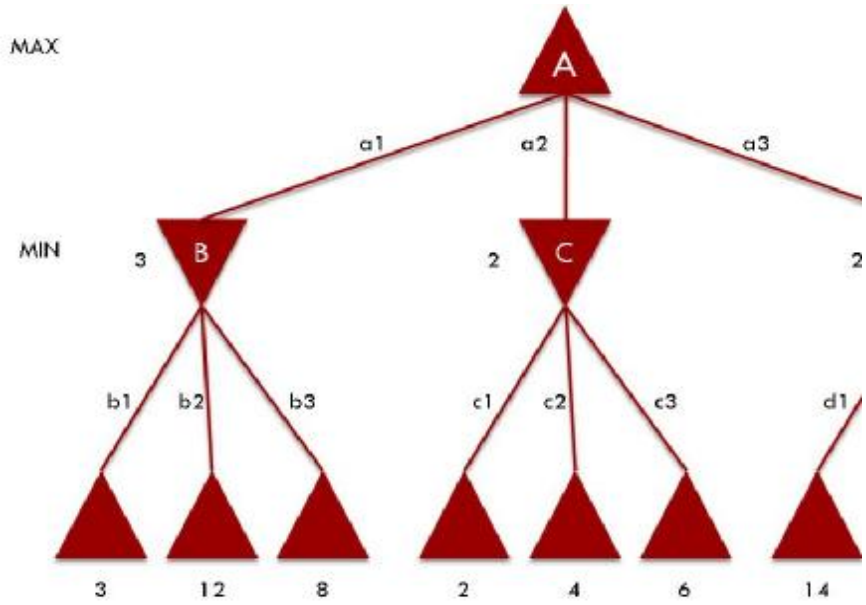
- Até mesmo o jogo da velha é complexo para traçarmos a árvore de jogo inteira em uma página e
  - assim, nos limitaremos ao jogo trivial da Figura a seguir.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com



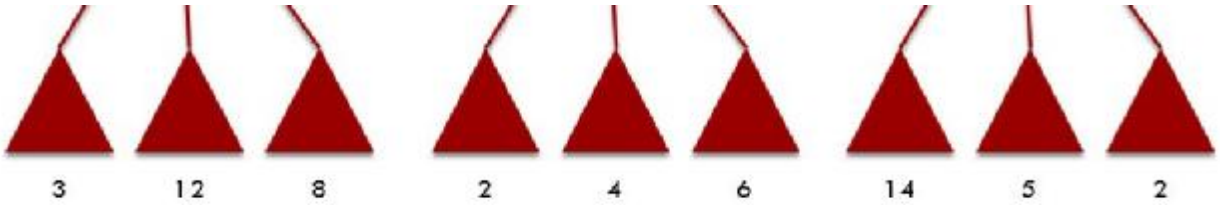
PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Os movimentos possíveis para MAX no nó raiz são identificados por *a1, a2 e a3.*
- As respostas possíveis para *a1* correspondentes a MIN são *b1, b2 e b3*.
- Esse jogo específico termina depois de um movimento realizado por MAX e por MIN.



PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- No linguajar dos jogos, dizemos que
  - essa árvore tem a profundidade de 1único movimento,
  - que consiste em 2 meios movimentos,
  - cada um dos quais é chamado jogada.)
- As utilidades dos estados terminais nesse jogo variam de 2 a 14.



PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Dada uma árvore de jogo,
  - a estratégia ótima pode ser determinada do valor minimax de cada nó
  - VALOR-MINIMAX(s).
- O valor minimax de um nó é
  - a utilidade (para MAX) de se encontrar no estado correspondente,
  - supondo-se que ambos os jogadores tenham desempenho ótimo
  - desde esse estado até o fim do jogo.
- É óbvio que o valor minimax de um estado terminal é simplesmente sua utilidade.

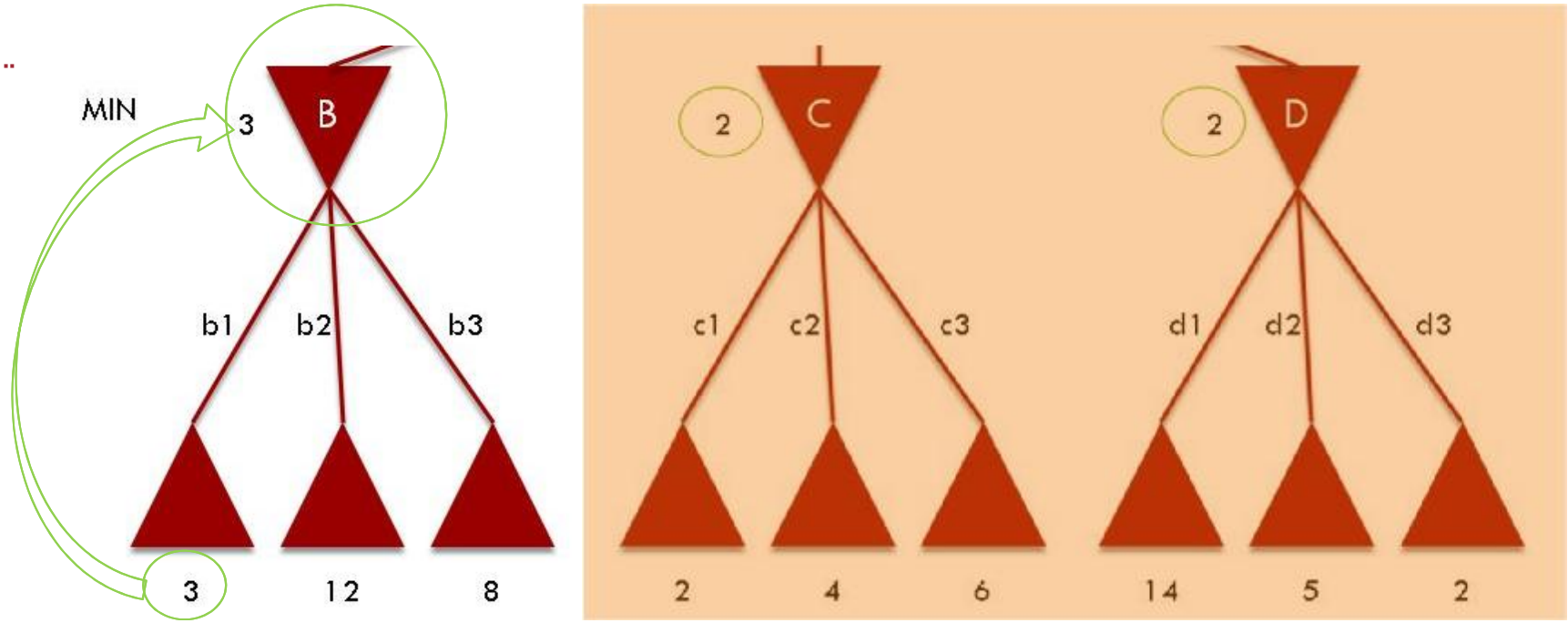
PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Além disso, dada uma escolha,
  - MAX preferirá se mover para um estado de valor máximo, enquanto
  - MIN preferirá um estado de valor mínimo.
- Assim, temos:

VALOR-MINIMAX(s) =

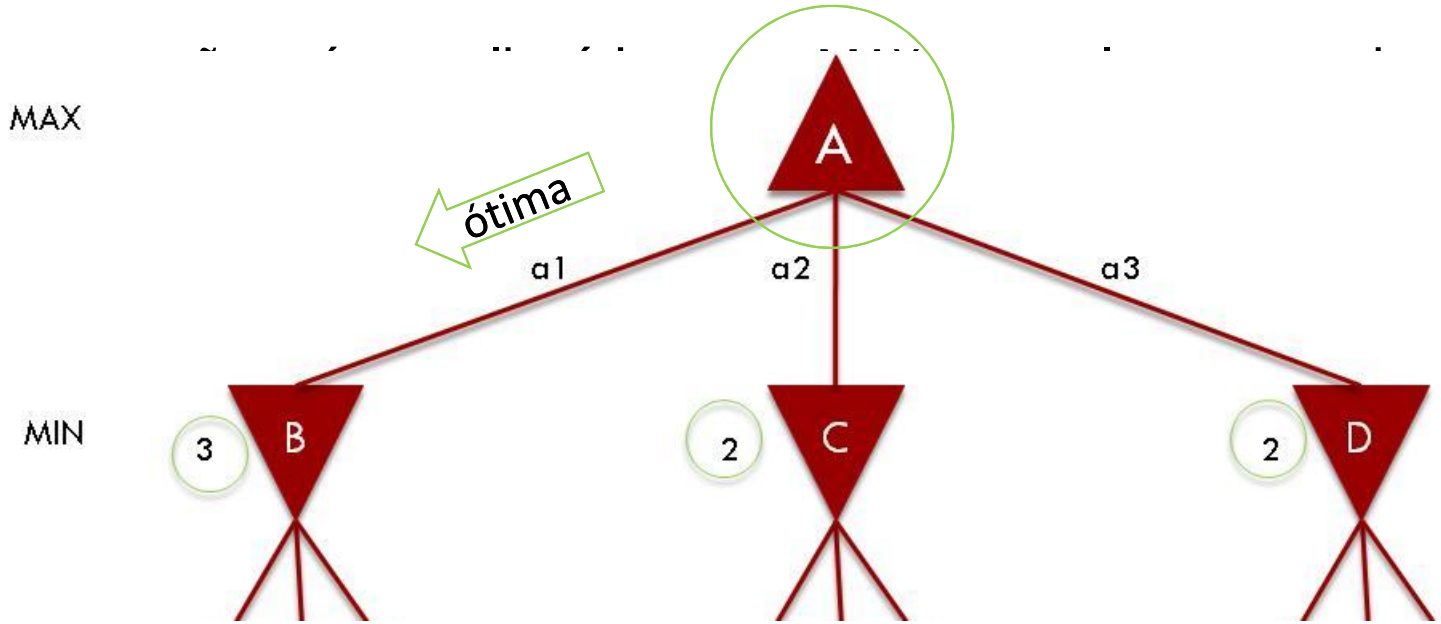
UTILIDADE(s)	se TESTE DE TÉRMINO(s)
Max a e Ações(s) MINIMAX(RESULTADO(s,a))	se JOGADOR(s) = MAX
Min a e Ações(s) MINIMAX(RESULTADO(s,a))	se JOGADOR(s) = MIN

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Os nós terminais no nível inferior obtiveram os valores utilidade da função UTILIDADE do jogo.
- O 1º nó de MIN, identificado por *B,* tem 3 sucessores com valores 3, 12 e 8; portanto seu valor minimax é 3.
- 

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- O nó raiz é um nó de MAX;
  - Seus sucessores têm valores minimax 3, 2 e 2;
  - Logo ele tem um valor minimax = 3
- Também podemos identificar a decisão minimax na raiz:



PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Essa definição de jogo ótimo para MAX supõe que MIN também jogue de forma ótima
  - ela maximiza o resultado para MAX no pior caso.
- E se MIN não jogar de forma ótima?
- Nesse caso é fácil mostrar que MAX terá um desempenho ainda melhor.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Outras estratégias que não sejam minimax podem:
  - funcionar melhor contra oponentes não ótimos,
  - mas têm um desempenho pior contra oponentes ótimos.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

5.2.1 O Algoritmo minimax

- O algoritmo minimax calcula a decisão minimax a partir do estado corrente.
- Ela utiliza uma computação recursiva simples dos valores minimax de cada estado sucessor,
  - implementando diretamente as equações da definição.
- A recursão percorre todo o caminho descendente até as folhas da árvore e,
  - depois os valores minimax são propagados de volta pela árvore,
  - á medida que a recursão retorna.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

função DECISÃO-MINIMAX(estado) retorna uma ação

retornar arg max a ∈ AçõES(s) de VALOR-MIN(RESULTADO(estado, a))

---

função VALOR-MAX(estado) retorna um valor de utilidade

se TESTE TERMINAL (estado) então retornar UTILIDADE(estado)

v ← + ∞

para cada a em AÇÕES(estado) faça

v ← MAX(v, VALOR-MIN(RESULTADO(s,a)))

retornar v

---

função VALOR-MIN(estado) retorna um valor de utilidade

se TESTE-TERMINAL (estado) então retornar UTILIDADE(estado)

v ← − ∞

para cada a em AÇÕES(estado) faça

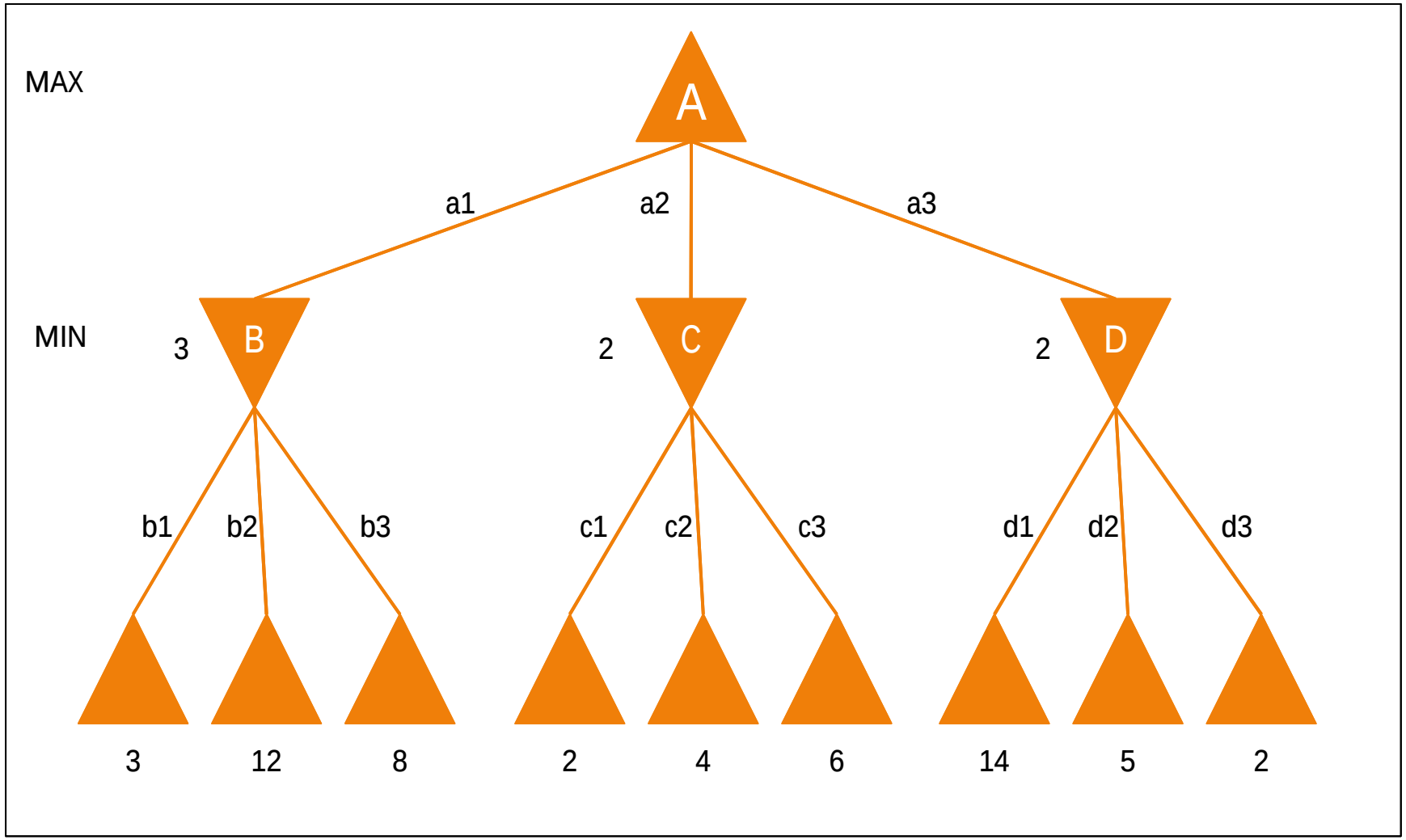
v ← MI N(v,VALOR-MAX(RESULTADO(s, a)))

retornar v

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- O algoritmo retorna a ação correspondente ao melhor movimento possível (maior utilidade)
  - Supondo o oponente que minimiza a utilidade.
- As funções VALOR-MAX e VALOR-MIN passam por toda a árvore de jogo, até as folhas,
  - A fim de determinar o valor de propagação de volta a um estado.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com



PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Ex: o algoritmo efetua uma recursão descendo a árvore até os 3 nós folhas inferiores e
  - usa a função UTILIDADE para descobrir que seus valores são 3, 12 e 8.
  - Em seguida, ele obtém o mínimo deles, 3,
    - e o devolve como valor propagado de volta para o nó *B*.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Um processo semelhante fornece os valores propagados de volta
  - de 2 para C
  - e 2 para D
- Por fim, tomamos o valor máximo entre 3, 2 e 2 para obter 3 e propagá-lo de volta para o nó raiz.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- O algoritmo minimax executa uma exploração completa em profundidade da árvore de jogo.
- Se a profundidade máxima da árvore é *m*
  - e existem *b* movimentos válidos em cada ponto,
  - a complexidade de tempo do algoritmo minimax é $O(b^m)$.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- A complexidade de espaço é $O(b^m)$ para um algoritmo que gera todos os sucessores de uma vez
  - ou $O(m)$ para um algoritmo que gera ações, uma de cada vez.
- É claro que,
  - em jogos reais, o custo de tempo é totalmente impraticável,
  - mas esse algoritmo serve como base para a análise matemática de jogos e para algoritmos mais práticos.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

Decisões ótimas em jogos com vários participantes

- Muitos jogos populares permitem mais de 2 jogadores.
- Vamos examinar a maneira de estender a ideia de minimax a jogos com vários jogadores.
- Isso é simples do ponto de vista técnico,
  - mas destaca algumas questões conceituais novas e interessantes.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Primeiro, precisamos substituir o único valor para cada nó por um vetor de valores.
- Por exemplo, em um jogo de 3 jogadores A, B e C, um vetor $< v_A , v_B , v_C >$ está associado a cada nó.
- Para os estados terminais, esse vetor fornece a utilidade do estado do ponto de vista de cada jogador.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

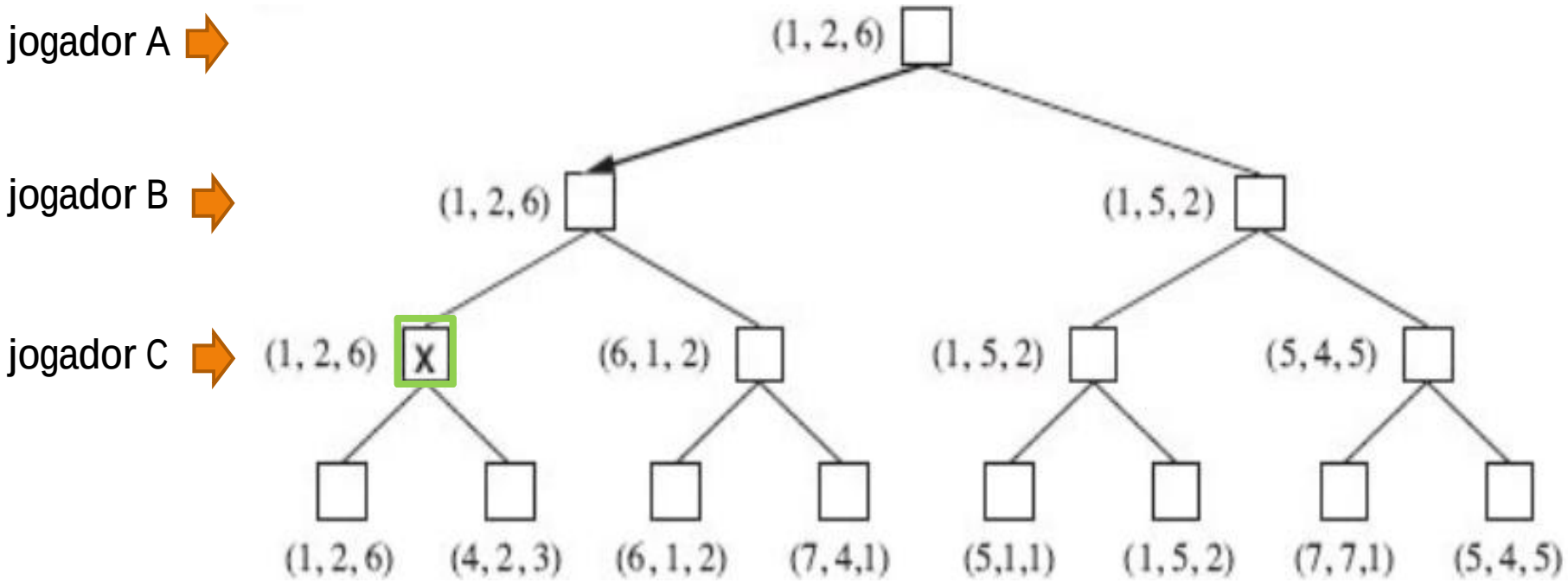
- obs: em jogos de soma zero com 2 jogadores, o vetor de 2 elementos pode ser reduzido a um único valor
  - porque os valores são sempre opostos.
- O caminho mais simples para implementar isso é fazer a função UTILIDADE retornar
  - um vetor de utilidades.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Agora temos de considerar
  - os estados não terminais.
- Vamos examinar o nó identificado por X na árvore de jogo da Figura a seguir.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Nesse estado, o jogador C define o que fazer.
- As 2 escolhas levam a estados terminais com vetores de utilidade
  - Mov. 1: $<v_A = 1, v_B = 2, v_C = 6>$ e
  - Mov. 2: $<v_A = 4, v_B = 2, v_C = 3>$
- 6 > 3, então X = Mov. 1 é melhor.



PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Jogos como o Diplomacy™,
  - com vários participantes,
  - normalmente envolvem alianças,
  - sejam elas formais ou informais.
- As alianças são feitas e desfeitas à medida que o jogo se desenrola.
  - Como entender tal comportamento?
- As alianças constituem uma consequência natural de estratégias ótimas para cada jogador em um jogo com vários participantes?
  - É possível que sim.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

Ex:

- Suponha que
  - A e B estejam em posições fracas
  - e que C esteja em uma posição mais forte.
- Então, em geral, é ótimo
  - A e B atacarem C,
  - para que C não destrua cada um deles.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- A colaboração emerge
  - de um comportamento puramente egoísta.
- Contudo, tão logo C enfraqueça com o violento ataque conjunto
  - A aliança perderá seu valor,
  - A ou B poderá violar o acordo.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Em alguns casos,
  - as alianças explícitas apenas tornam concreto
  - aquilo que teria acontecido de qualquer modo.
- Em outros,
  - Um estigma social incorpora-se ao romper uma aliança
  - Eles devem buscar o equilíbrio entre
    - a vantagem imediata de romper uma aliança
    - e a desvantagem a longo prazo de serem considerados pouco confiáveis.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Se o jogo for de soma diferente de zero,
  - a colaboração também poderá ocorrer
  - com apenas 2 jogadores.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Ex: vamos supor que exista um estado terminal com utilidades
  - $<v_A = 1000, v_B = 1000>$
  - e que 1000 seja a mais alta utilidade possível para cada jogador.
- Então, a estratégia ótima é
  - os 2 jogadores fazerem todo o possível para alcançar esse estado,
  - cooperarem de forma automática para atingir uma meta mutuamente desejável.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

5.3 Poda alfa-beta

- O problema da busca minimax é que o nº de estados de jogo que ela tem de examinar é
  - exponencial em relação ao nro de movimentos,
  - não podemos eliminar o expoente,
  - mas podemos reduzí-lo pela metade.
- O artifício é a possibilidade de
  - calcular a decisão minimax correta
  - sem examinar todos os nós da árvore de jogo.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

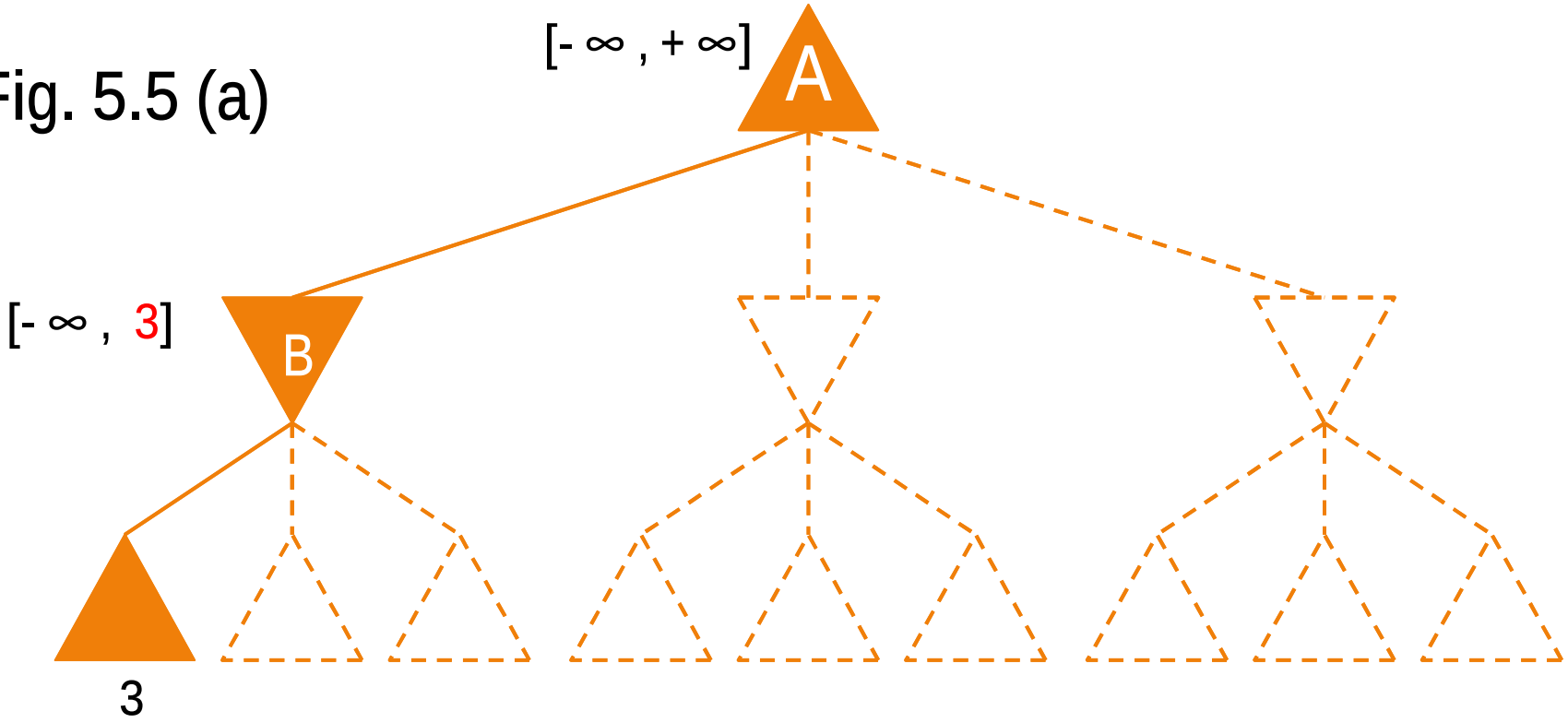
- Podemos usar a ideia de poda alfa-beta,
  - a fim de eliminar grandes partes da árvore.
- Quando a poda é aplicada a uma árvore minimax padrão,
  - ela retorna o mesmo movimento que minimax retornaria,
  - mas poda as ramificações que não terão influência possível sobre a decisão final.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Considerando a árvore da figura 5.2
- O foco será o que se conhece em cada ponto do processo.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

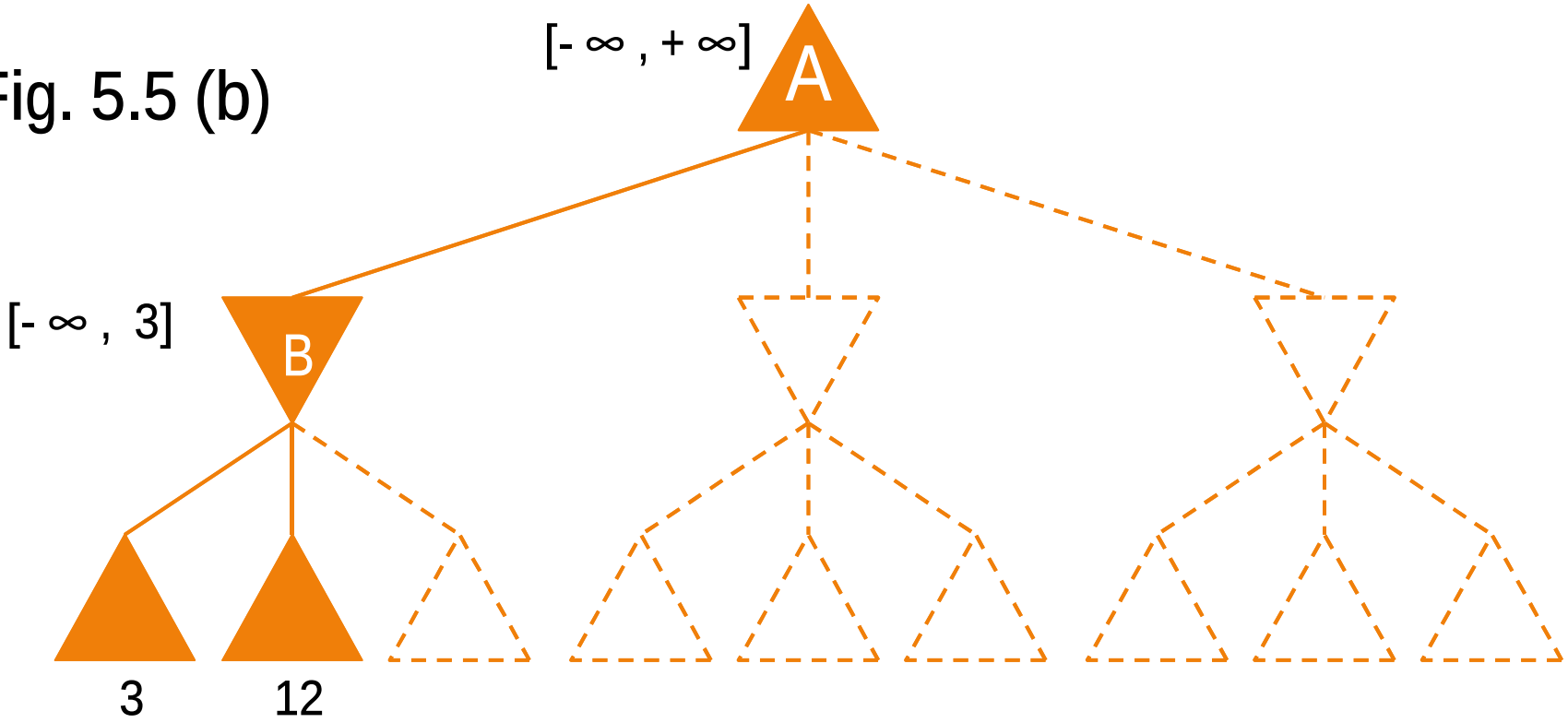
Fig. 5.5 (a)



(a) A primeira folha sob B tem valor 3.
Consequentemente, B, que é um nó de MIN, tem valor máximo 3.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

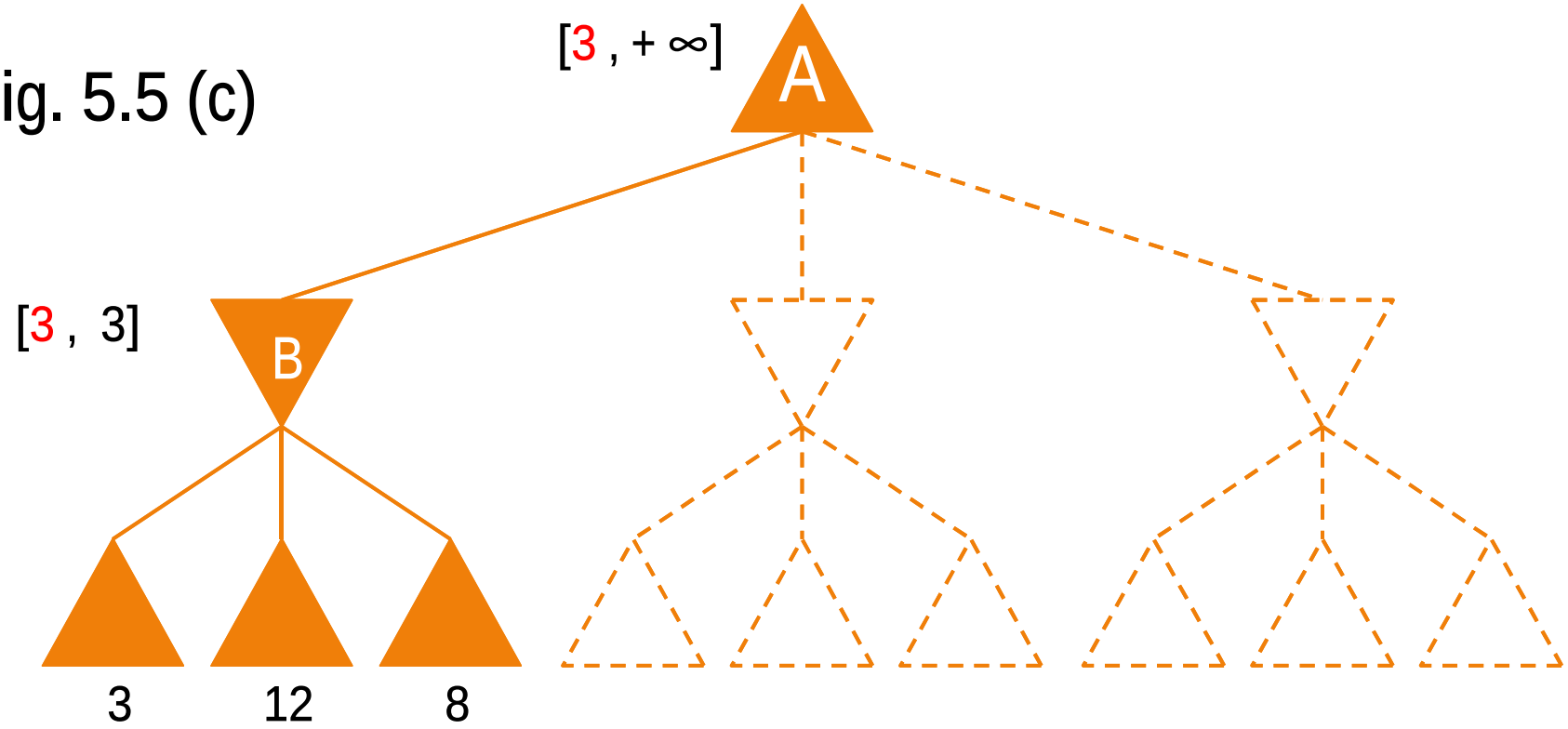
Fig. 5.5 (b)



(b) A segunda folha sob B tem valor 12;
MIN evitaria esse movimento,
de modo que o valor de B ainda é, no máximo, 3.

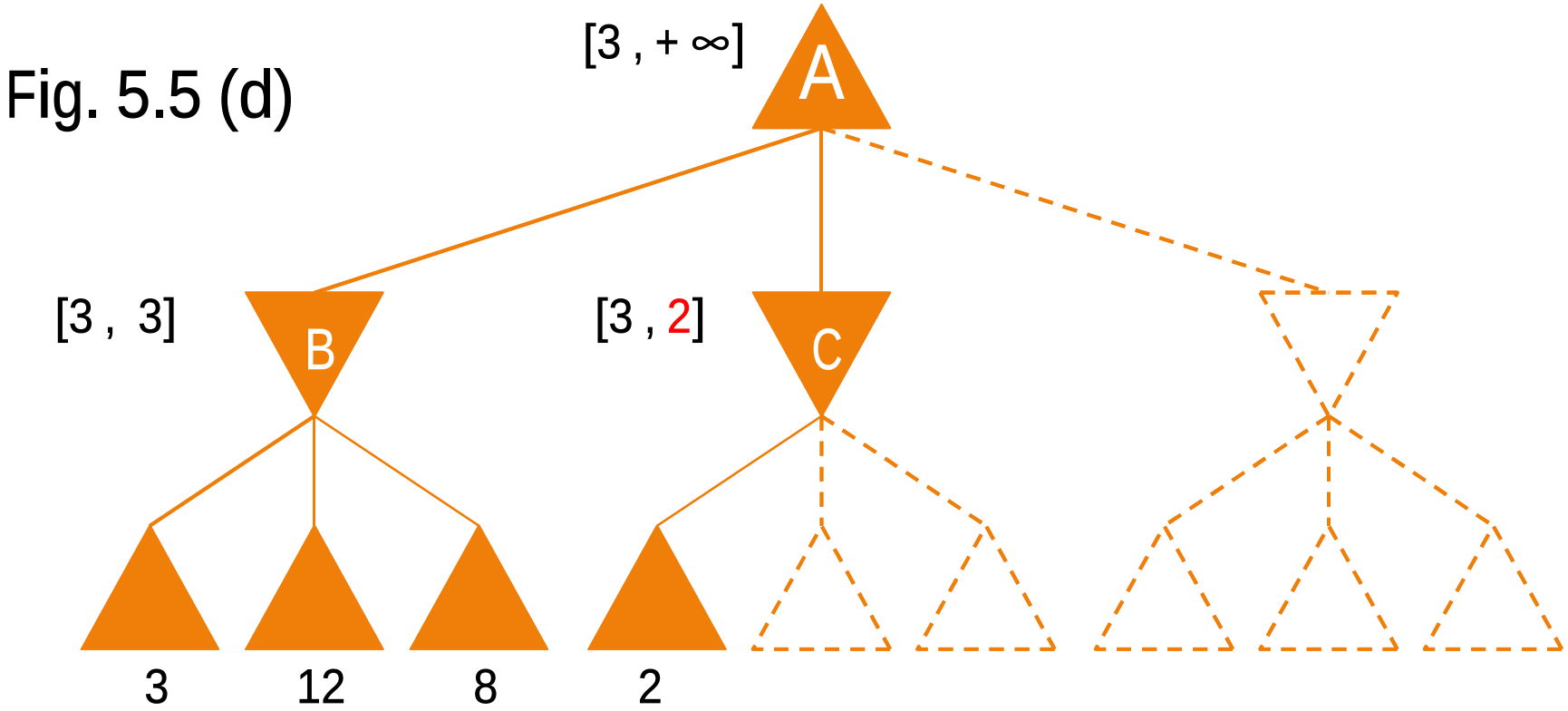
PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

Fig. 5.5 (c)



(c) A terceira folha sob B tem valor 8; vimos todos os estados sucessores de B e, assim, o valor de B é exatamente 3.
Agora, podemos deduzir que o valor da raiz é pelo menos 3, porque MAX tem uma escolha de valor 3 na raiz .

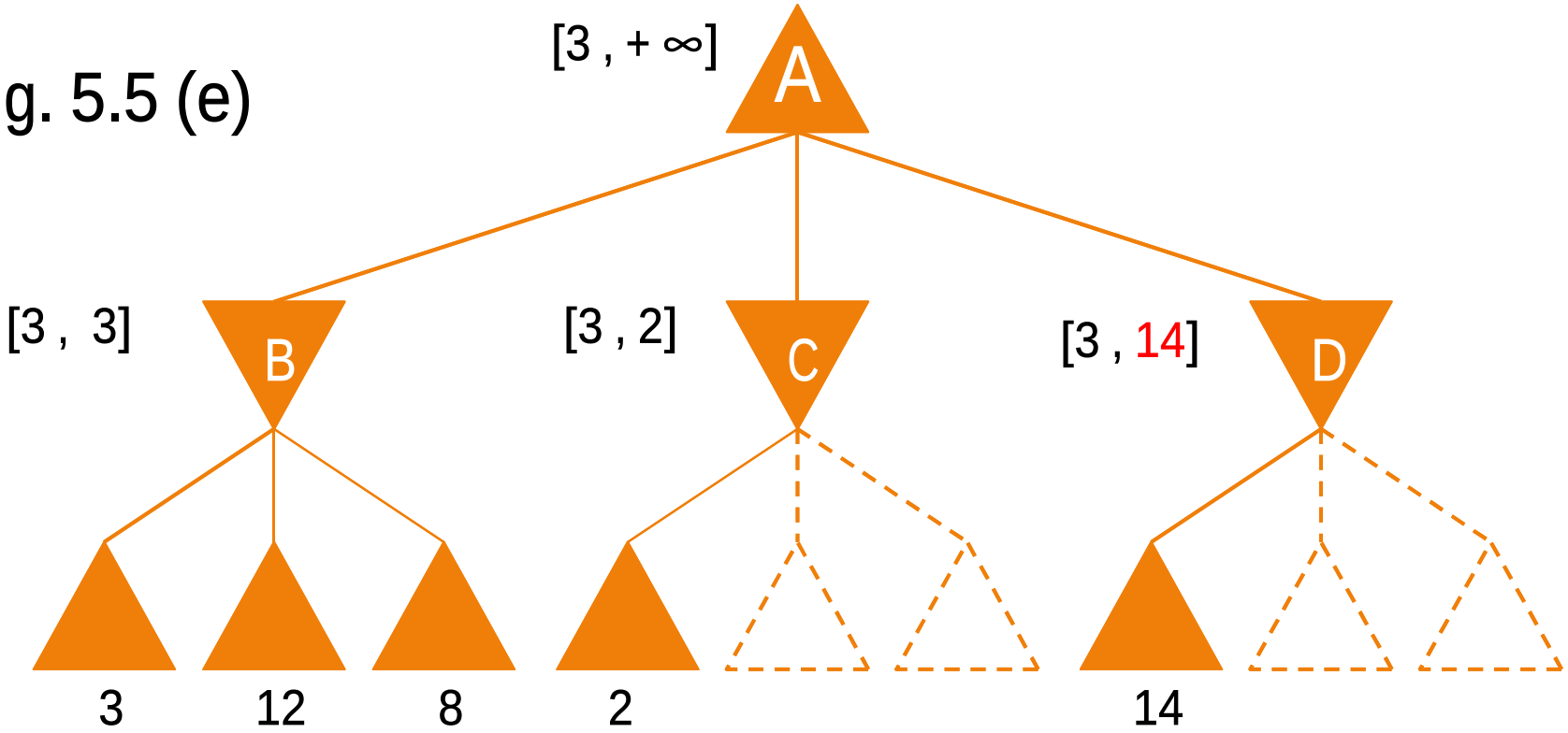
PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com



(d) A primeira folha abaixo de C tem o valor 2. Consequentemente, C, que é um nó de MIN, tem valor máximo 2. Porém, sabemos que B vale 3; portanto, MAX nunca escolheria C. Desse modo, não há razão para se examinar os outros sucessores de C. Esse é um exemplo de poda alfa-beta.

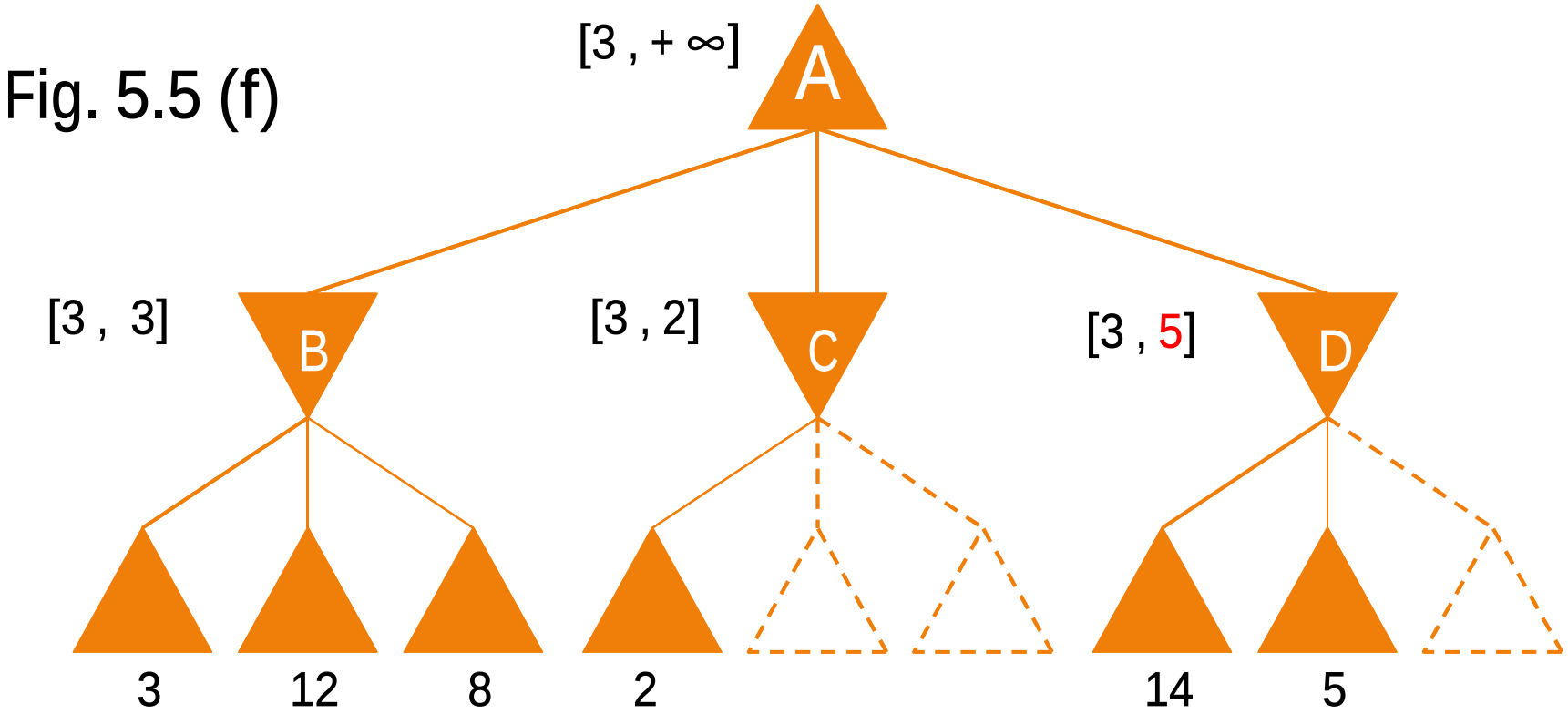
PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

Fig. 5.5 (e)



(e) A primeira folha abaixo de D tem o valor 14, e então D vale no máximo 14. Esse valor ainda é mais alto que a melhor alternativa de MAX (isto é, 3) e, portanto, precisamos continuar a explorar sucessores de D. Note também que agora temos limites para todos os sucessores da raiz e, consequentemente, o valor da raiz também é no máximo 14.

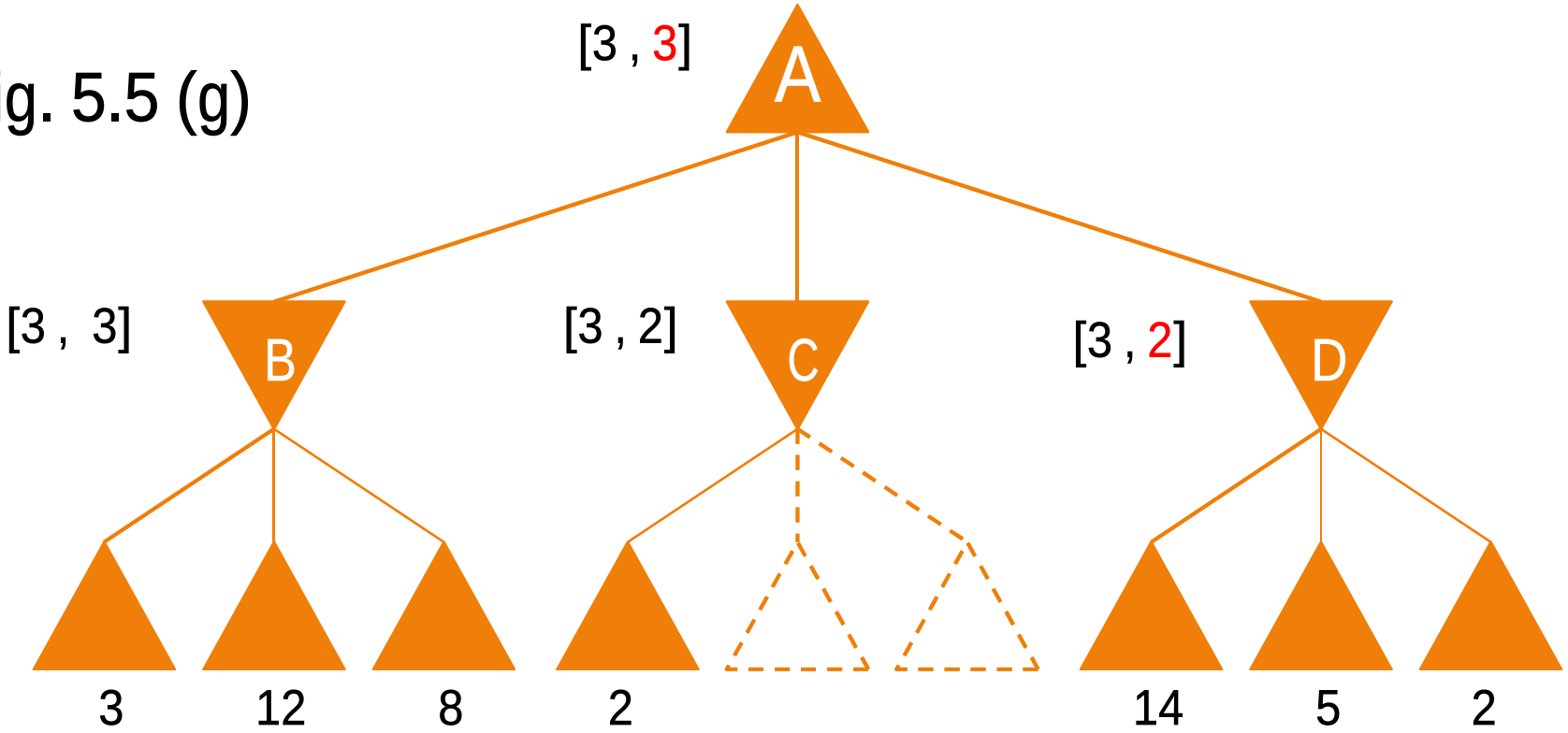
PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com



(f) O segundo sucessor de D vale 5 e, assim, novamente precisamos continuar a exploração.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

Fig. 5.5 (g)



(g) O terceiro sucessor vale 2; agora, D vale exatamente 2. A decisão de MAX na raiz é efetuar o movimento para B, o que nos dá o valor 3.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Isto também pode ser visto como uma simplificação da fórmula de VALOR-MINIMAX.
  - Sejam *x* e *y* valores dos 2 sucessores não avaliados do nó *C* nas Figuras anteriores
  - e seja $z = \min(2, x, y)$
  - Então, o valor do nó raiz é dado por:

$$\text{MINIMAX}(raiz) = max(min(3, 12, 8), min(2, x, y), min(14, 5, 2))$$

$$= max(3, min(2,x,y), 2)$$

$$= max(3, z, 2) \qquad onde\ z = min(2, x, y) <= 2$$

$$= 3$$

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Em outras palavras,
  - o valor da raiz e,
  - consequentemente, a decisão minimax
  - são independentes dos valores das folhas podadas *x* e *y*.
- obs:
  - a poda alfa-beta pode ser aplicada a árvores de qualquer profundidade e
  - frequentemente é possível podar subárvores inteiras no lugar de podar apenas folhas.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- O princípio geral é este:
  - Considere um nó *n* em qualquer lugar na árvore, tal que o jogador tenha a escolha de movimento até esse nó.
  - Se o jogador tiver
    - uma escolha melhor *m* no nó pai de *n*
    - ou em qualquer ponto de escolha adicional acima dele,
    - então *n nunca será alcançado* em um jogo real.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

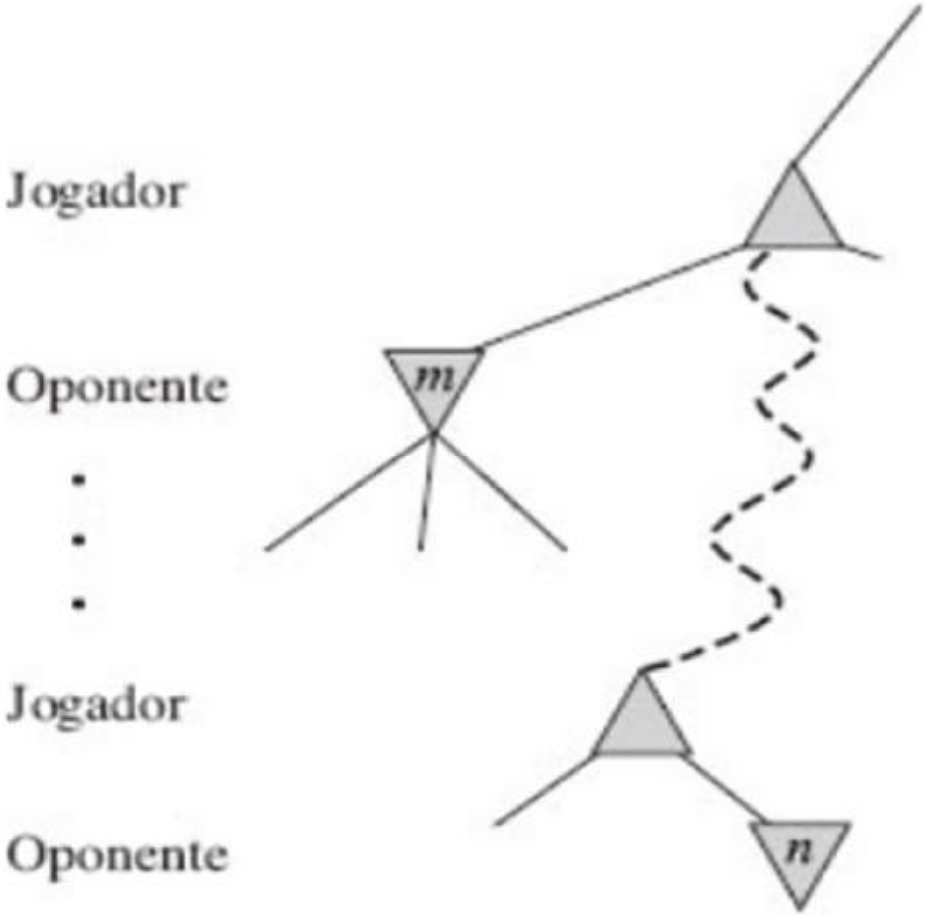


Figura 5.6

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Assim,
  - uma vez que descobrimos o suficiente sobre *n* (examinando alguns de seus descendentes)
  - para chegar a essa conclusão,
  - poderemos podá-lo.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- A busca minimax é em profundidade então
  - em qualquer instante
  - só temos de considerar os nós
  - ao longo de um único caminho na árvore.
- A poda alfa-beta obtém seu nome a partir dos 2 parâmetros a seguir
  - que descrevem limites
  - sobre os valores propagados de volta
  - que aparecem em qualquer lugar ao longo do caminho:

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- **α** = o valor da melhor escolha (valor mais alto) para MAX
  - que encontramos até o momento em qualquer ponto da escolha ao longo do cominho.
- **β** = o valor da melhor escolha (valor mais baixo) para MIN
  - que encontramos até agora em qualquer ponto de escolha ao longo do caminho.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- A busca alfa-beta atualiza os valores de α e β
  - á medida que prossegue e poda as ramificações restantes em um nó;
  - e encerra a chamada recursiva;
  - tão logo se sabe que o valor do nó corrente é pior que o valor corrente de α ou β para MAX ou MIN.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

O algoritmo completo é mostrado a seguir.

função BUSCA-ALFA-BETA(*estado*) retorna uma ação

$v \leftarrow$ VALOR-MAX(*estado*, $-\infty$, $+\infty$)

retornar *ação a* em AÇÕES(*estado*) com valor *v*

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

função VALOR-MAX(*estado*, α, β) retorna *um valor de utilidade*

se TESTE-TERMINAL (estado)

então retornar UTILIDADE(estado)

$v \leftarrow -\infty$

para cada *a* em AÇÕES(estado) faça

$v \leftarrow$ MAX(v, VALOR-MIN(RESULTADO(s, a), α, β))

se $v \geq \beta$ então retornar v # *poda*

$\alpha \leftarrow$ MAX(α, v)

retornar v

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

```
função VALOR-MIN(estado, α, β) retorna um valor de utilidade
    se TESTE-TERMINAL(estado)
        então retornar UTILIDADE(estado)
    v ← +∞
    para cada a em AÇÕES(estado) faça
        v ← MIN(v, VALOR-MAX(RESULTADO(s, a), α, β) )
        se v ≤ α então retornar v     # poda
        β ← MIN(β, v)
    retornar v
```

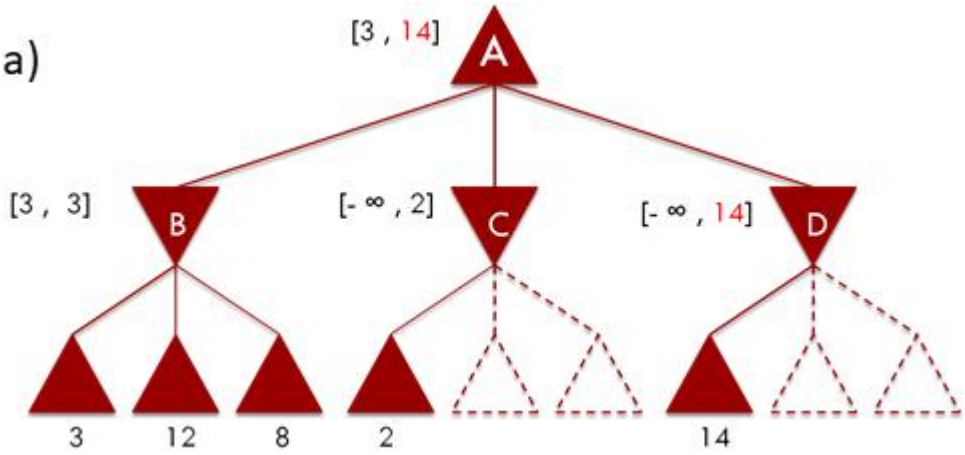
PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

5.3.1 Ordenação de movimentos

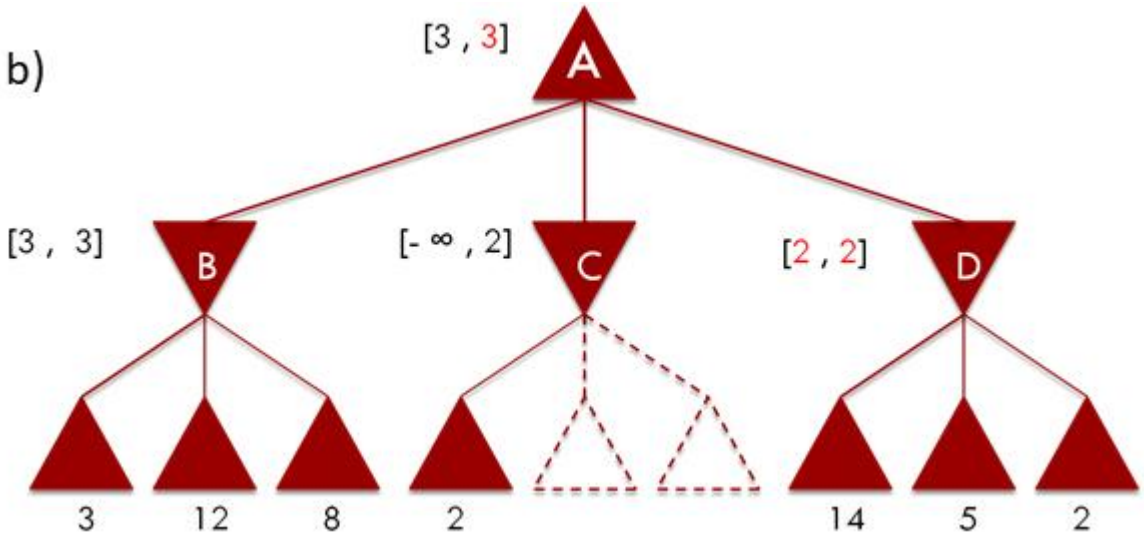
- A efetividade da poda alfa-beta é
  - altamente dependente da ordem em que os estados são examinados.
- Ex: na Fig. a seguir a) e (b), não poderíamos podar quaisquer sucessores de *D*
  - Porque os piores sucessores (de MIN) foram gerados primeiro (14 e 5).
  - Se o 3º sucessor tivesse sido gerado primeiro, poderíamos podar 14 e 5.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

a)



b)



PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Isso sugere que poderia valer a pena tentar examinar primeiro
  - os sucessores que têm probabilidade de serem melhores.
- Se supor-se que isso possa ser feito:
  - alfa-beta precisará examinar $O(b^{m/2})$ nós
  - ao invés de (b^m).

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- A ramificação efetiva será de $\sqrt{b}$ em vez de b
- No caso do xadrez, 6 em vez de 35.
- Em outras palavras,
  - alfa-beta poderá resolver uma árvore
  - aproximadamente 2 vezes tão profunda
  - como minimax no mesmo tempo.
- Se os sucessores forem examinados em ordem aleatória,
  - o nº total de nós examinados será de $O(b^{3m/4})$
  - para um valor moderado de *b*.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- No caso do xadrez, uma função de ordenação bastante simples
  - como capturas primeiro,
  - depois ameaças,
  - depois movimentos à frente e,
  - em seguida, movimentos para trás
- levará a uma distância de aproximadamente 2 vezes o resultado do melhor caso, $O(b^{m/2})$.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Acrescentar esquemas dinâmicos de ordenação de movimentos,
  - como tentar primeiro os movimentos considerados os melhores da última vez,
  - nos levará até bem perto do limite teórico.
- O passado pode ser o lance anterior
  - muitas vezes, as mesmas ameaças permanecem
  - ou poderia vir da exploração do lance atual.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Uma maneira de obter informações do lance atual é com busca de aprofundamento iterativo de pesquisa.
  - 1º pesquise uma jogada profunda e registre o melhor caminho de movimentos.
  - 2º busque uma jogada mais profunda, mas utilize o caminho registrado para informar a ordenação do movimento.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- O aprofundamento em uma árvore de jogo exponencial
  - acrescenta apenas uma fração constante para o tempo total de busca,
  - mais do que compensado por uma melhor ordenação de movimento.
- As melhores jogadas são muitas vezes chamadas de lances mortais
  - e tentá-los de primeira é chamado de
  - heurística de lance mortal.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Estados repetidos na árvore de busca podem causar um aumento exponencial no custo da busca.
- Em vários jogos, estados repetidos ocorrem com frequência devido a transposições
  - permutações diferentes da mesma sequência que terminam na mesma posição.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- Ex:
  - se as brancas têm um movimento a_1 que pode ser respondido pelas pretas com b_1
  - e um movimento não relacionado a_2 no outro lado do tabuleiro que pode ser respondido por b_2
  - as sequências $[a_1 , b_1 , a_2 , b_2]$ e $[a_1 , b_2 , a_2 , b_1]$ terminarão na mesma posição.
- Vale a pena armazenar a avaliação da posição resultante em um tabela hash.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

- A tabela hash de posições já vistas é tradicionalmente chamada de tabela de transposição.
  - O uso de uma tabela de transposição pode ter um efeito drástico,
    - chegando às vezes a duplicar a profundidade de busca acessível no xadrez.
  - Mas, se estivermos avaliando 1 milhão de nós por segundo, não será prático manter todos na tabela de transposição.
  - São usadas diversas estratégias para escolher que nó fica e que nó sai da tabela.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com

Bibliografia

- RUSSEL, Stuart Jonathan; NORVIG, Peter. Inteligência Artificial. Tradução por Regina Célia Simille, Artificial intelligence, 3rd ed. Rio de Janeiro: Elsevier, 2013.

PDF criado com pdfFactory Pro versão de avaliação www.pdffactory.com